AI Engineer Intelligence Pipeline — Architecture

GraphOne / FrontierAtlas demo task · production design summary (≤3 pages)

1. Scale Strategy — collecting 500,000+ startups, products, and papers without manual intervention

The system is built as a source-adapter + bounded-worker-pool pattern (`src/crawlers/base.py`), not a linear script. Each source (ArXiv, Papers with Code, YC directory, Product Hunt, etc.) implements `BaseSourceAdapter.discover()` — an async, paginated generator of `CrawlItem`s — and `fetch_and_parse()`. An `AsyncWorkerPool` fans this out across N concurrent workers with a bounded queue (backpressure), file/Redis-backed checkpointing (resumability — a killed run restarts without reprocessing successes), and per-item outcomes (SUCCESS/RETRYING/FAILED/SKIPPED/INVALID/STALE/DUPLICATE) so nothing is silently lost.

Going from this trial's ~3,000 records to 500,000+ requires zero adapter or orchestration code changes — only:

- Raising `concurrency` (worker count) per source, bounded by each source's own rate limits.
- Horizontally scaling worker containers (the queue + checkpoint design is safe for multiple processes/nodes; dedup coordination across nodes is handled by the fingerprint + Redis SETNX layer described in §3).
- Pagination cursors already being part of the adapter contract, so "more records" is a config change (target count / page depth), not a new code path.
- PostgreSQL handling the write volume via batched upserts on the UNIQUE constraints already defined per table.

2. Handling 413 (Payload Too Large) and 429 (Rate Limited) across thousands of concurrent extractions

413 strategy (`src/llm/token_budget.py`): before every LLM call we strip boilerplate (nav/cookie/newsletter text, deduplicated repeated lines), estimate tokens (chars/4 heuristic), and compute a hard input budget = provider context window − reserved output tokens − reserved system tokens. If the cleaned text still exceeds budget we split it into paragraph-bounded chunks (never mid-sentence), rank chunks by an information-density score (alnum-token ratio + presence of digits, which correlates with specs/prices/dates), and always prepend the title + source URL so entity names survive truncation. We never blindly truncate from the end.

429 strategy (`src/llm/retry.py`): exponential backoff with full jitter (`random.uniform(0, min(base·2^attempt, cap))`), honoring a provider's `Retry-After` header when present instead of guessing. A per-provider `CircuitBreaker` opens after 5 consecutive failures and imposes a cooldown, so a hammered provider gets relief instead of every worker retrying it in lockstep (the exact failure mode that causes 429 storms). On exhaustion, the `ExtractionOrchestrator` falls through the configured chain — Gemini Flash → Groq Llama 3 → DeepSeek — trying the same chunk on the next provider rather than failing the record outright. This path was live-verified against a real 429 from the GitHub REST API during this build (shared sandbox IP hit GitHub's 60 req/hr unauthenticated limit); the breaker opened exactly as designed.

All raw LLM output is JSON-parsed (with a best-effort salvage for prose-wrapped JSON) and Pydantic-validated before acceptance; a schema mismatch is treated as an extraction failure, never coerced into a guess.

3. Freshness Tracking — never processing the same article/job twice across distributed crawler nodes

Every news/job item is reduced to a deterministic fingerprint: `sha256(normalize_url(url) + '|' + normalize_title(title) + '|' + published_date_iso)` (`src/freshness/fingerprint.py`). URL normalization strips tracking params (utm_*, fbclid, gclid) and trailing slashes so mirrors/syndicated copies collapse to the same key; combining URL + title + date guards against URL-only false negatives (same URL, edited title) and title-only false positives (generic titles across different articles).

Coordination across nodes uses Redis `SETNX` as an atomic distributed claim (`src/freshness/dedup.py`) — whichever node claims a fingerprint first proceeds, every other node immediately sees `DUPLICATE` and skips without a fetch. A PostgreSQL `UNIQUE` constraint on the fingerprint column is the second line of defense, so correctness never depends on Redis being up — only throughput does. The strict 24-hour freshness gate itself (`src/extraction/date_parser.py::is_within_last_24h`) additionally requires the parsed publication date to

carry confidence ≥ 0.6; low-confidence relative-date guesses (e.g. bare "month"/"year" units) are excluded from the strict output rather than risked.

4. Storage Strategy

Primary store: PostgreSQL. The domain is fundamentally relational — startup → product, paper → GitHub repo, company → job/news — and we need transactional, constraint-enforced idempotent writes (the UNIQUE constraints in `src/storage/models.py` are the actual dedup enforcement, not just documentation). JSONB columns absorb the parts of the schema that evolve as LLM extraction improves, without a migration per tweak.

Vector layer: pgvector (extension, not a separate service) for entity-embedding similarity — used as a second-pass suggestion source feeding into the resolver's controlled fuzzy-match stage, never as the sole basis for merging two entities. Keeping vectors inside Postgres avoids a second database with its own consistency story for a workload that doesn't yet need a dedicated vector DB's scale.

Graph relationships: modeled relationally (foreign-keyed junction tables), not a separate graph database. At this entity/relationship density (5 record types, a handful of relationship kinds) a graph engine like Neo4j adds operational surface area without a capability Postgres foreign keys + recursive CTEs don't already cover; it's the kind of addition the brief explicitly warns against making "merely to make the README look impressive." If multi-hop graph queries (e.g. "papers by authors who also founded a startup in our seed list") become a first-class product need, that's the trigger to introduce Neo4j — not before.

Cache/coordination: Redis for dedup claims and cross-worker rate-limit signaling. Ephemeral by design — losing Redis loses throughput headroom, never correctness (Postgres UNIQUE constraints are the source of truth).